

AI-SOC-Assistant Evolution & Wins

A concise engineering brief showing how the project matured into a reproducible, explainable, and safety-conscious SOC triage pipeline.

Capability	Professional signal
5-class family classification	Separates Normal, DoS, Probe, R2L, and U2R instead of returning only a binary anomaly flag.
Stable dual-detector triage	Combines supervised attack probability with a training-calibrated Isolation Forest risk signal.
Validated SHAP evidence	Reports real feature values and whether each contribution supports or opposes the predicted class.
Governed ticket generation	Defaults to a deterministic offline renderer and safely validates optional LLM output.

Engineering Evolution

1. Robust Feature Encoding and Class Balance

Categorical protocol, service, and flag fields use `OneHotEncoder(handle_unknown="ignore")`. Class imbalance is handled only on the training fold through deterministic duplication of exact rows, so categorical, binary, and integer telemetry is never interpolated.

Win: Novel categories no longer crash inference, and balanced training data remains valid network telemetry.

2. Stable Anomaly Calibration

Isolation Forest calibration is learned once from training-normal traffic and reused for batch and single-record scoring. The fused score uses the probability of any non-normal class rather than the classifier's maximum confidence.

Win: Alert selection and ticket generation now agree, independent of unrelated test-batch rows.

3. LangGraph Orchestration with Pinned Compatibility

The ReAct agent uses the `state_modifier` prompt API supported by the pinned LangGraph 0.2.53 stack, with direct provider integrations for Ollama, OpenAI, and Anthropic.

Win: Every supported provider can construct the agent graph without a version-signature failure.

4. Local-First Network Boundary

Template mode makes no network calls, and the default Compose service runs with networking disabled. Ollama keeps prompts on the configured local server; cloud providers are explicitly external. AbuseIPDB and NVD tools remain off unless `SOC_ENABLE_THREAT_INTEL=true` is deliberately set.

Win: Privacy claims now match the actual deployment boundary and require explicit opt-in for external lookups.

Safety, Explainability, and Evaluation

5. Threat-Intelligence Correctness and Input Safety

IP addresses and service names are validated before use. API calls have timeouts and malformed-response handling. NVD searches use a bounded 119-day publication window and sort results newest first.

Win: Unsafe inputs are rejected, and API failures cannot prevent deterministic incident-ticket generation.

6. Class-Aware SHAP Semantics

Predicted labels are mapped through the fitted model's classes_ ordering, feature schemas must match exactly, and contribution directions are described as supporting, opposing, or neutral for the predicted class.

Win: Explanations remain correct for non-contiguous labels and avoid unsupported causal attack claims.

7. Evidence-Bound GenAI Guardrails

Only allow-listed detector evidence reaches the LLM. Returned tickets must preserve validated detector semantics and evidence, use safe source-bound SPL queries, and omit raw SHAP values. Any malformed, unsafe, or provider-failed response falls back to the deterministic renderer.

Win: The optional LLM acts as a governed presentation layer rather than an authority over incident facts.

8. Protocol-Scoped, Reproducible Evaluation

Holdout and KDDTest+ cross-distribution evaluations write separate artifacts. The current results report 99.88% holdout accuracy and 74.40% cross-distribution accuracy, exposing the real generalization gap. Dataset checksums, supported Python versions, and a shared-code notebook make reruns auditable.

Win: Strong in-distribution performance is presented alongside honest rare-class and distribution-shift limits.

Validation snapshot

Clean Python 3.12 dependency install, full automated tests, formatting, lint, dependency consistency, default and no-balancing pipeline runs, and both evaluation protocols pass. Container build and non-root checks are enforced in CI across Python 3.10-3.12.

Portfolio Positioning

The repository now demonstrates not only model training, but also production-minded evidence handling, network boundaries, dependency hygiene, regression coverage, and honest evaluation. Those qualities make the project easier to operate, review, and extend in its next implementation phase.

Updated 10 August 2026